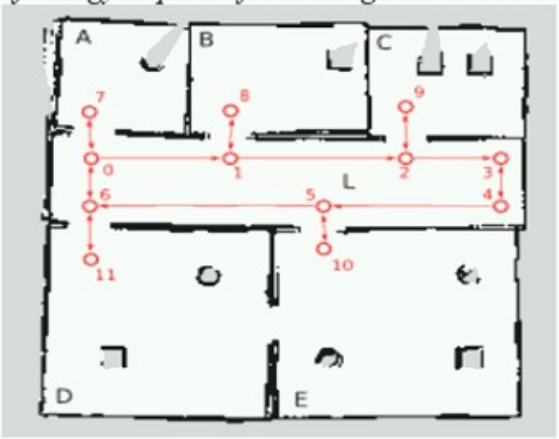
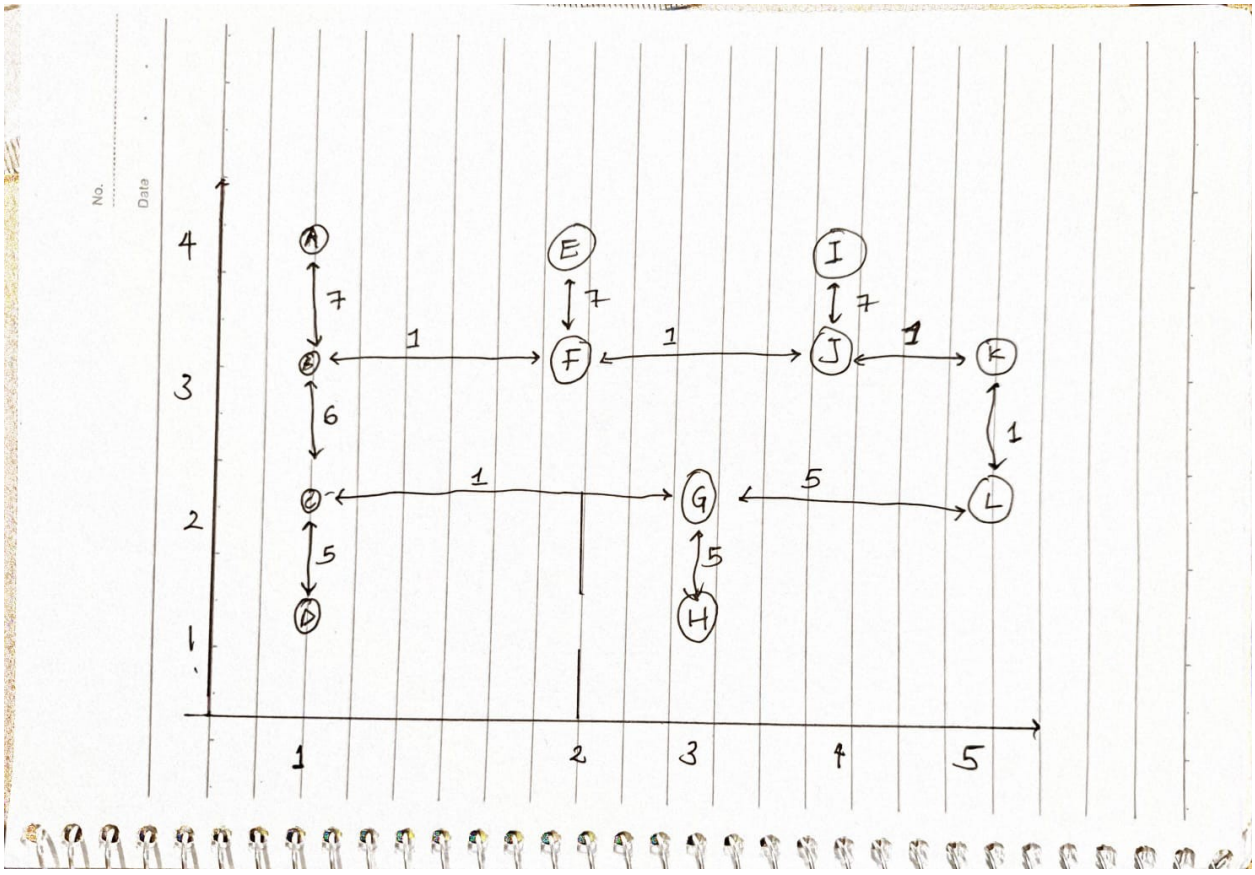
The map of a building is provided below with different points (red circles) where a surveillance robot can scan a room from. The difference between the values of any two points shows the amount of energy required for moving the robot between the two points.





by:

Harina Chohan

```
import operator
class environment:
    my_graph = {"A":set(["B"]),
                "B":set(["C","F", "A"]),
                "C":set(["B", "G", "D"]),
                "D":set(["C"]),
                "E":set(["F"]),
                "F":set(["B","E","J"]),
                "G":set(["L","C", "H"]),
                "H":set(["G"]),
                "I":set(["J"]),
                "J":set(["F","I", "K"]),
                "K":set(["J", "L"]),
                "L":set(["K","G"]),

               }
    my_heuristics = {"A" : ["1", "4"],
                     "B" : ["1", "3"],
                     "C" : ["1", "2"],
                     "D" : ["1", "1"],
                     "E" : ["2", "4"],
                     "F" : ["2", "3"],
                     "G" : ["3", "3"],
                     "H" : ["1", "3"],
                     "I" : ["4", "4"],
                     "J" : ["4", "3"],
                     "K" : ["5", "3"],
                     "L" : ["5", "2"]
                    }

    cost = {str(["A", "B"]):"7",
            str(["B", "A"]):"7",
            str(["B", "C"]):"6",
            str(["C", "B"]): "6",
            str(["C", "D"]): "5",
            str(["D", "C"]):"5",
            str(["B", "F"]): "1",
            str(["F", "B"]): "1",
            str(["F", "E"]):"7",
            str(["E", "F"]): "7",
            str(["F", "J"]): "1",
            str(["J", "F"]):"1",
            str(["I", "J"]):"7",
            str(["J", "I"]): "7",
            str(["K", "J"]): "1",
            str(["J", "K"]):"1",
            str(["L", "K"]): "1",
            str(["K", "L"]): "1",
            str(["L", "G"]):"5",
            str(["G", "L"]): "5",
```

```
            str(["G", "H"]): "5",
            str(["H", "G"]):"5",
            str(["C", "G"]): "1",
            str(["G", "C"]): "1"
           }
    start = "A"
    goal = "L"

class agent(environment):
    def GBFS(graph,start, goal):
        p=[]

        p.append(start)
        while True:
            neighbour = graph[start]
            heur={}
            for i in neighbour.difference(p):
                l=[]
                l.append(str(start))
                l.append(str(i))
                heur[i] = agent.getheur(i,goal)
            sorted_heur = sorted(heur.items(),
key=operator.itemgetter(1))
            x=next(iter(sorted_heur[0]))
            p.append(x)
            if x == goal:
                return p
            else:
                start=x

    def A_STAR(graph,start, goal):
        p=[]

        p.append(start)
        while True:
            neighbour = graph[start]
            heur={}
            for i in neighbour.difference(p):
                l=[]
                l.append(str(start))
                l.append(str(i))
                heur[i] = agent.getheur(i,goal)+agent.get_cost(l)
            sorted_heur = sorted(heur.items(),
key=operator.itemgetter(1))
            x=next(iter(sorted_heur[0]))
            p.append(x)
            if x == goal:
                return p
            else:
                start=x
```

```
    def get_cost(path_to_cost):
        i=0
        path_cost=0
        while i < len(path_to_cost)-1:
            l=[]
            l.append(path_to_cost[i])
            l.append(path_to_cost[i+1])
            path_cost=path_cost+int(environment.cost[str(l)])
            i+=1
        return path_cost

    def getheur(vertex, goal):
     v=[]
     g=[]
     for i in environment.my_heuristics[vertex]:
        v.append(int(i))
     for i in environment.my_heuristics[goal]:
        g.append(int(i))
     heuristic = abs(v[0]-g[0])+abs(v[1]-g[1])
     return heuristic

    def __init__(self,environment):
        print("the path with the lowest battery
consumption:",agent.A_STAR(environment.my_graph,environment.start,envi
ronment.goal))

e1=environment()
a1=agent(e1)
```

```
A_STAR: ['A', 'B', 'F', 'J', 'K', 'L']
```